

A Memory-Flow View of MoE Inference: Bank-Aware Virtualization and Prefetching for On-Chip Accelerators

摘要

本文的逻辑不是直接说 MoE 是 memory-bound，而是先从专家负载和专家规模不均推出 MoE 产生动态 Bank-level memory-flow；再通过 Bank-conflict-aware 模型证明这种动态访存流在静态 Bank 划分下会造成 Bank hotspot、Bank idle 和 Stall；最后提出 MemDomain，通过统一访存路径、虚拟 Bank 动态映射和 Bank-aware 预取协同，分别解决资源割裂、固定绑定和预取冲突问题。

I. Introduction

- MoE 成为大模型扩展的重要结构
- MoE 的 token-to-expert routing 会造成专家负载不均；专家负载不均进一步导致 IA / Weight / OA 的片上访存流随时间变化；这种动态访存流与静态 Bank 划分不匹配，最终造成 Bank hotspot、Bank idle、Bank conflict 和 memory stall。
- 现有工作不足：毕设照搬（补充 ISCA20265 篇文章为什么不行）
- 提出 MemDomain，动态资源分配和预取的协同优化
- 创新点罗列

II. Background and Motivation

II.1 Motivation Experiment

- 验证访存瓶颈主要集中在 MoE 上
- 验证访存瓶颈出现的原因归因于静态分配
- 普通预取可能会导致新的 bank 冲突问题

II.2 Bottleneck Analysis

Challenge:

1. 专家分布不均导致内存需求不同
2. 静态策略不能匹配动态数据流
3. 普通预取可能会导致新的 bank 冲突问题

II.3 问题为什么难解决，引出 MemDomain

III. MemDomain Architecture

III.1 overview

- 介绍一下 bb
- 简要讲一下前中后端的实现

III.2 Acc/SRAM 的总线统一

- 叙述如何实现总线统一
- 原本割裂的片上存储访问资源放到同一套调度逻辑下，使普通 SRAM 访问和累加写回都能参与统一 Bank 管理，为后续动态映射提供统一入口

III.3 虚拟资源到物理资源的映射

- 叙述如何实现映射
- 说明虚拟 Bank 映射可以把软件看到的逻辑 Bank 和硬件实际物理 Bank 解耦，使同一个逻辑数据对象在不同运行阶段映射到不同物理 Bank，从而根据当前 memory-flow 动态选择空闲或低冲突 Bank **为什么可以解决 Bank 调度的问题的主要内容！**

III.4 预取协同优化

- 叙述如何实现预取协同
- **为什么协同优化能进一步降低访存延时**

IV. Evaluation

4.1 实验准备

- 介绍实验平台
- 介绍工作负载测试网络
- 介绍硬件配置
- 介绍对比方案和命名规则
- 介绍评价指标

4.2 动态 Bank 虚拟化机制评估

- 验证动态分配能否缓解 Bank 冲突

4.3 预取与动态 Bank 分配的消融实验

- 验证预取和动态 Bank 分配是否存在协同效果

4.4 预取窗口和 Chunk 粒度敏感性实验 (new!)

- 观察不同预取窗口和不同的 chunk 粒度会带来什么样子的结果

4.5 路由分布与专家配置敏感性实验 (new!)

- 验证方案对不同 MoE 路由模式的鲁棒性
- TOP-k, 专家数量和输入的改变下动态是否还有好结果

4.6 DC

4.7 总结

V. Related Work

5.1 MoE 路由优化工作

5.2 MoE 通信优化

5.3 动态预取协同优化在多核的情况如何

VI. Conclusion